

# An Improved Linux Priority Scheduling Method Based on XGBoost

Ningke Yang  
School of Computer Science  
and Technology  
Xi'an University of  
Posts and Telecommunications  
Xi'an, China

Xinfeng Shu  
School of Computer Science  
and Technology  
Xi'an University of  
Posts and Telecommunications  
Xi'an, China

Chen Liang  
School of Computer Science  
and Technology  
Xi'an University of  
Posts and Telecommunications  
Xi'an, China

**Abstract**—In order to address the issue of excessive resource consumption due to frequent context switching when running a large number of compute-intensive tasks in high-load scenarios on Linux. The paper proposes a scheduling algorithm that utilizes eBPF programs to extract fine-grained data such as process memory usage, execution time, read/write rates, etc. This data is then used to construct a dataset, and an XGBoost algorithm model is employed to predict the future resource consumption of processes. Simultaneously, based on the characteristics of binary classification and multi-classification tasks in the XGBoost algorithm, tasks are assigned different scheduling priorities according to CPU utilization rates. Tasks of corresponding types are then executed based on CPU status before the priority scheduling algorithm is applied. Experimental comparisons reveal that under similar conditions, reducing the overhead of context switching results in a 1.03% improvement in overall task execution time and enhances CPU running efficiency. Consequently, it is concluded that this improved Linux priority scheduling method based on XGBoost can enhance execution efficiency when running numerous compute-intensive tasks in high-load environments.

**Keywords**—eBPF, XGboost, High-load, Operating system, Task classification.

## I. INTRODUCTION

With the continuous development of computer systems and networks, process scheduling in large-scale computer systems has become key to improving performance and energy efficiency. However, in the face of high loads, process resource prediction and task classification bring about quite complex issues. Traditional methods often rely on manually designed rules and experiences, which are unable to cope with increasingly complex and dynamic computing environments. Particularly in environments with frequent context switches, the performance overhead is significant. Building upon this, leveraging eBPF for fine-grained data extraction is proposed. eBPF, a technology for executing secure, efficient, programmable kernel and user-space code within the Linux kernel, enables more accurate data extraction due to its placement in kernel space, flexible filtering rules, rich feature support, and low-overhead execution.

A Linux process can be characterized by a sequence of system calls. Therefore, analyzing the system call sequence of a process can reveal its behavioral pattern[1], [2]. Process

resource prediction is a technique used to forecast the amount of resources that processes or applications in a computer system may require in future time periods. Make more accurate and effective decisions in resource allocation and scheduling to enhance system performance, resource utilization, and user experience.

Context switching is the fundamental mechanism for multitasking scheduling in real-time and embedded systems. This is a highly resource-intensive process, significantly impacting the usage of CPU, memory, and other system resources[3]. Context switching incurs performance overhead on threads in a multitasking environment. Due to its intricate operations, it also incurs significant overhead, resulting not only in noticeable performance degradation of applications but also in a substantial decrease in system efficiency[4], [5]. To minimize context switching and enhance system performance, effective process resource detection and task classification are required, laying the groundwork for energy conservation[6], [7].

The traditional rule-based approaches, both domestically and internationally, have some limitations in process resource prediction and task classification. They typically rely on predefined rule sets, making it difficult to adapt to new security threats, and require a significant amount of effort in rule writing and maintenance. Currently, these two tasks primarily rely on rules and experience. However, the drawback of this traditional approach is its heavy reliance on manual intervention. Moreover, when dealing with complex, dynamic, and large-scale data, its performance often falls short of requirements. Therefore, more advanced model algorithms are needed[8]. The current mainstream methods include using algorithms such as random forest and gradient boosting trees for prediction[9], as well as models like Long Short-Term Memory (LSTM) and Convolutional Neural Networks (CNN).

The challenges and issues facing process load methods domestically and internationally go beyond what has been mentioned above. Especially under high-load conditions, process load prediction still faces additional challenges and

issues[10]. Firstly, the intense utilization of system resources under high load may result in more complex process behaviors, increasing the unpredictability of process behavior. This necessitates the design of more flexible and adaptive models to capture this complexity. Secondly, process data in high-load environments often exhibit hidden characteristics and variability. The same is true for task classification. The research on task classification domestically and internationally lacks dynamic adjustment mechanisms. Task classification often occurs at system startup, yet as the system operates, both task types and load conditions may change. Existing task classification methods lack dynamic adjustment mechanisms, failing to adapt in real-time to changes in tasks, resulting in decreased effectiveness in task scheduling and management. It is necessary to consider the correlation between tasks in task classification to ensure tasks are scheduled and executed reasonably.

Load prediction can assist systems in managing and utilizing computer resources more intelligently, enhancing resource utilization, performance, and reliability. By predicting system load, better planning and optimization of system resource allocation can be achieved. In high-load environments, the accuracy of collecting relevant process data directly impacts the accuracy of the process's CPU utilization rate. eBPF can be utilized to implement efficient system tracing and diagnostic tools, such as Dynamic Tracing and performance analysis tools. This involves inserting eBPF programs into the kernel to capture system events and performance metrics. eBPF programs need to be inserted into specific locations within the kernel to execute, extracting relevant information such as process I/O rates, memory usage, and runtime. These programs enable highly granular data collection and analysis, better catering to specific requirements, thereby making system monitoring and performance tuning more flexible and precise.

To address the aforementioned issues, leveraging more advanced feature representation methods on top of extracting finer-grained datasets using eBPF, powerful classification algorithms such as deep learning-based feature extraction techniques and XGBoost models are employed to capture and predict resource consumption patterns[11]. XGBoost is utilized to establish the connection between behavioral features and corresponding risk levels, and key features are identified through feature importance ranking and recursive elimination. In this scenario, utilizing CPU affinity can effectively reduce the performance overhead caused by context switching. By executing related tasks on the same processor, frequent thread and context switches can be avoided, thus enhancing the system's operational efficiency. By utilizing the XGBoost model to predict operations on the dataset, followed by task categorization based on the predicted values, and adjusting task priorities according to the current CPU status for scheduling execution, the aim is to enhance execution efficiency and save execution time.

TABLE I  
INDICATOR INFORMATION

| Indicators          | Type   | Unit |
|---------------------|--------|------|
| CPU usage           | float  | %    |
| Memory usage        | float  | %    |
| Process read speed  | double | kb/s |
| Process write speed | double | kb/s |
| Queue length        | int    |      |

## II. DATASET CONSTRUCTION

This paper focuses on monitoring and analyzing process data. Because accurate prediction of future resource consumption relies on training models on current process data, eBPF programs need to be inserted into kernel breakpoints to obtain finer-grained process resource consumption data. Based on this, the dataset is constructed through integration. Constructing datasets for process data enables training machine learning models to identify process behaviors and understand resource utilization. The dataset primarily involves data extraction and cleaning through eBPF. Depending on experimental requirements, the dataset collects information such as I/O scheduling, memory size, CPU usage, memory read/write rates, and memory utilization from 200 to 500 processes of various scales.

### A. Data collection

In high-load environments, certain processes may consume a significant amount of CPU resources, such as computationally intensive tasks, loops, or recursive operations. In such high-load scenarios, these processes may lead to CPU resource contention, resulting in system slowdowns or the inability of other processes to execute. Additionally, processes exhibiting memory leaks, I/O blocking, deadlock situations, and other resource-consuming anomalies may occur in high-load environments. Therefore, the following metrics are selected to detect the resource utilization of processes in high-load environments. The information for each metric is shown in Table I.

Extended Berkeley Packet Filter (eBPF) is an instruction set and execution environment within the Linux kernel. It enables runtime modification, interaction, and kernel programmability. eBPF primarily [12] runs user-specified programs within the kernel, thereby providing a flexible and powerful monitoring and analysis approach. The successor Extended BPF (eBPF) enhances flexibility and is implemented through a virtual machine with a just-in-time (JIT) compiler and an interpreter running within the kernel [13]. eBPF programs need to be inserted into specific positions within the kernel to execute. This paper primarily utilizes kprobes, tracepoints, and uprobes technologies to extract relevant information such as CPU usage, memory consumption, and runtime of processes. In high-load environments, utilizing CPU usage for process prediction is a common approach [14].

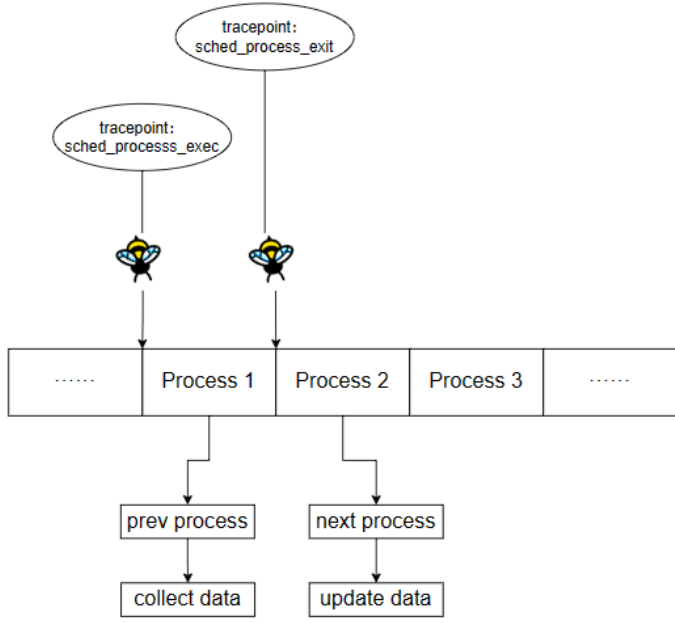


Fig. 1. eBPF program data collection solution

The data collection is achieved by attaching a tracepoint program to the kernel function `sched-process-exit`. When a process begins execution, this function is executed. Upon the termination of process execution, another kernel function, `sched-process-exit` is executed to mark the end of process execution. The function `'bpf-ktime-get-ns()'` is used to collect statistical information about the process, including its start time, end time, process name, PID, PPID, runtime duration, CPU usage rate, memory usage, and other information. This data is stored in a BPF Map. The schematic diagram of eBPF kernel space program data collection is illustrated in Fig 1.

Data processing Due to the increased computation resources, time, and memory usage associated with using floating point numbers in the Linux kernel, and to minimize unnecessary memory consumption, calculations for each process's usage time, CPU usage rate, memory usage rate, and process read/write speed are performed in the user space program of eBPF. In the user space, a total duration of 200 seconds has been set, which can be adjusted as needed. At the same time, in the user space eBPF program, a time period has set to retrieve practical information about all process resources in the BPF at each interval. Because CPU usage and memory utilization are related to the total amount, after the completion of the 200s total duration, the CPU usage and memory utilization will be calculated. Then, the information will be printed using the csv program for further operations. The CPU usage of a process can be calculated using the following equation:

$$P_{process\_cpu} = \frac{process\_cpu\_use\_time}{all\_time} * 100\% \quad (1)$$

In this equation,  $P_{process\_cpu}$  represents the CPU utilization

of a process,  $all\_time$  represents the time interval for data collection, and  $cpu\_use\_time$  represents the execution time of the process during that time period, including both user space and kernel space execution time. The memory usage of a process can be calculated as follows:

$$P_{process\_mem} = \frac{process\_memused}{memtotal} * 100\% \quad (2)$$

In this equation,  $P_{process\_mem}$  represents the memory utilization of a process,  $process\_memused$  represents the physical memory usage of the process, and  $memtotal$  represents the total size of the system memory. The read/write speed of a process can be calculated as follows:

$$P_{process\_read\_rate} = \frac{process\_read\_char}{process\_cpu\_use\_time} / 1024 \quad (3)$$

$$P_{process\_write\_rate} = \frac{process\_write\_char}{process\_cpu\_use\_time} / 1024 \quad (4)$$

### B. Data cleaning

Even though eBPF programs can provide relatively accurate data, there are still some issues such as duplicate values or erroneous data. It requires performing data cleansing operations to remove some unusable data. The following are three methods for related operations:

**Design code to filter duplicate values, keeping only one instance of each identical value in the current MAP.** During the data cleansing process, measures can be taken to eliminate these duplicate values, ensuring that each numerical value is retained only once. This simplifies the dataset, reduces redundancy, and enhances the accuracy and efficiency of data analysis and model construction.

**Remove erroneous values.** Similarly, design code to exclude data with incorrect data types or values that exceed the expected range. These erroneous data values may lead to bias and errors in model construction and data analysis.

**Adjusting data format.** As we plan to later utilize the XGBoost algorithm for data processing, a script will be developed to modify the data format to meet the algorithm's requirements. This ensures compatibility with the XGBoost algorithm.

### C. Task classification design

In high-load environments, task classification aids system administrators or monitoring systems in identifying and addressing issues. By monitoring and analyzing the CPU usage of tasks, anomalies can be detected and addressed promptly, ensuring the system's smooth operation. In high-load environments, tasks can be classified into the following categories:

**real-time tasks:** Real-time tasks: Real-time tasks refer to those that need to be completed within a specific time frame and have strict requirements regarding completion time.

These tasks typically need to be completed at predetermined time points or within certain time periods.

**Resource-Intensive tasks.** Resource-intensive tasks: Resource-intensive tasks are those that require high CPU usage to function properly. These tasks typically involve processing large amounts of data, performing complex calculations, or executing other computationally intensive operations. In high-load environments, these tasks may consume a significant amount of system resources, potentially restricting the operation of other tasks.

In classification problems, XGBoost can be used as a classification model. Classification problems involve assigning input samples to different categories, such as spam email classification or disease diagnosis. In classification problems, XGBoost utilizes decision trees as base learners and trains and predicts using gradient boosting. The difference lies in the application of specific loss functions [15] (such as the logistic loss function for binary classification) to optimize the model in classification problems.)

The first part of the model corresponds to the case where the true label is 1, using the negative log-likelihood function to measure the difference between the predicted probability and the true label. The second part corresponds to the case where the true label is 0, also using the negative log-likelihood function for measurement. By minimizing this loss function, the model's predicted probabilities can be made as close as possible to the true labels. The logistic loss function for binary classification is as follows:

$$L^{(t)} = \sum_{i=1}^n y_i \log(1 + \exp(-y_i^t)) + (1 - y_i) \log(1 + \exp(-y_i^t)) \quad (5)$$

### III. PROCESS RESOURCE PREDICTION BASED ON XGBOOST MODEL

XGBoost (eXtreme Gradient Boosting) is a machine learning algorithm based on Gradient Boosting Trees, consisting of multiple CART (Classification And Regression Tree) models. It improves and optimizes upon the Gradient Boosting Tree algorithm, offering high efficiency[16] and excellent predictive capabilities. The core idea is the Boosting concept, which involves iteratively training multiple weak classifiers (typically decision trees) and then combining them to form a strong classifier. At each iteration, the fitting target of the current model is adjusted based on the prediction results of previous models, enabling the next model to better correct the errors of the previous ones.

XGBoost's work in resource prediction primarily focuses on the following aspects:

**Ability to handle high-dimensional sparse data:** XGBoost can effectively handle high-dimensional sparse data

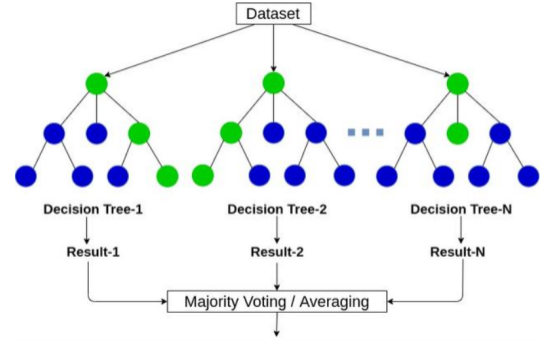


Fig. 2. Conceptual diagram of a CART-based model

by techniques such as feature column weighting and missing value handling, enhancing model stability and accuracy.

**Handling small amounts of outlier data amidst large volumes of normal data:** XGBoost can better address the classification issues between small amounts of outlier data and large volumes of normal data by improving its capability to handle class imbalance problems.

**Strong robustness:** XGBoost enhances model robustness and predictive capabilities further by employing ensemble learning with multiple decision trees.

Training an XGBoost model involves using normal data as training samples to construct the model. During the training process, XGBoost iteratively optimizes the objective function, generates multiple decision trees, and updates the weights[17] and parameters of the leaf nodes. The model is as depicted in Fig 2.

CART assumes that decision trees are binary trees, where internal node features are binary ("yes" or "no"). Such a decision tree is equivalent to recursively partitioning each feature, dividing the input space or feature space into a finite number of units, and determining the predicted value or category on these units.

The core algorithm adopts the Boosting idea, employing a forward-additive model where multiple trees collectively make decisions. The result of each tree is the difference between the target value and the predictions of all previous trees, and all results are cumulatively added to obtain the final result.

In the training set, there are a total of  $n$  points, each assigned a weight  $W_i$  to represent its importance. Through sequential model training, the weights of the points are adjusted. If a point is classified correctly, its weight is decreased; if classified incorrectly, its weight is increased. Initially, all weights are equal. Each misclassified point's weight is slightly increased before the next classification. This approach results in a classifier that performs well on both test and training data.

The XGBoost algorithm conforms to the following model function, where K refers to the number of trees, i.e., K base learners:

$$f_{m(x)} = \sum_{k=1}^K f_k(x_i) = y_i \quad (6)$$

The XGBoost algorithm mainly consists of three processes:

**Construct objective function:** For this algorithm, the first step is to define the objective function to measure the performance of the model. In GBDT, the objective function is the loss function  $L(y_i, y)$ . In XGBoost, an additional regularized term is introduced to control the structure of the base learner tree. The objective function is defined as follows:

$$obj = \sum_{i=1}^n L(y_i, y) + \sum_{k=1}^K \Omega(f_k) \quad (7)$$

The loss function representing the model is the loss between the true values and predicted values for each  $\sum_{i=1}^n L(y_i, y)$  represents the loss function of the model, which measures the discrepancy between the true values and predicted values for each sample.sample.

$\sum_{k=1}^K \Omega(f_k)$  represents the complexity of a tree, which can be evaluated by the number of layers or the number of leaf nodes in the tree.The purpose of constructing an objective function is to assess the quality of the model, so in an ideal situation, a lower value of the objective function is preferred. Since XGBoost is a boosting framework algorithm that involves additive training, the predicted value of the Kth model is equal to the sum of the predicted values of the previous K-1 models plus the predicted value of the current Kth model being trained. Simultaneously, the algorithm model y is used, and the complexity of subsequent base learner trees is decomposed into the sum of the complexities of the previous K-1 trees and the complexity of the current model. As the current Kth model is being trained, the complexities of the previous models are treated as constants. Thus, the objective function is modified as follows:

$$obj = \min \sum_{i=1}^n L(y_i, y_i^{k-1} + f_k(x_i)) + \Omega(f_k) \quad (8)$$

where  $y_i^{k-1}$  represents the prediction values of the previous K-1 tree models.

**Expanding the function using a second-order Taylor series.**This step involves simplifying equation8 by performing a Taylor expansion and simplifying the constants in the equation. According to the description[18] in the relevant literature, we obtain equation??.

$$obj = \min \sum_{i=1}^n [g_i f_k(x_i) + \frac{1}{2} h_i f_k^2(x_i)] + \Omega(f_k) \quad (9)$$

TABLE II  
RELATED INFORMATION

| Item   | Description                                        |
|--------|----------------------------------------------------|
| CPU    | Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz 2.80 GHz |
| Memory | 8GB DDR4                                           |
| Disk   | 1TB SATA Disk                                      |
| OS     | Ubuntu20.04                                        |
| Kernel | Linux5.19                                          |

**Parameterizing the objective function and introducing the tree structure into the objective function.** Parameterizing the objective function and introducing the tree structure into the objective function.To incorporate decision trees into the objective function, XGBoost employs a tree-based regularization method, which minimizes the sum of the squares of the weights of the tree's leaf nodes. Here,  $W_j$  represents the weight of the jth leaf node, T represents the number of trees, and  $\gamma$  and  $\lambda$  are the regularization parameters. This regularization term can be expressed as:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (10)$$

#### IV. THE ALGORITHM IMPROVEMENT IS DESCRIBED AS FOLLOWS

This paper primarily utilizes the preemptive priority scheduling algorithm in Linux, while also optimizing it based on the XGBoost algorithm model.

Priority scheduling algorithms can be applied to both high-level scheduling (job scheduling) and low-level scheduling (process scheduling), and can also be used in real-time systems. In high-level scheduling, the priority scheduling algorithm selects the job with the highest priority from the back-up job queue, assigns it with corresponding resources, and creates a process to be added to the process ready queue. In low-level scheduling, the priority scheduling algorithm selects the process with the highest priority from the process ready queue and assigns CPU resources to it for execution. If there are multiple jobs/processes with the highest priority, a first-come, first-served or shortest job/shortest process scheduling strategy can be combined.

#### V. EXPERIMENT

##### A. Experimental Environment

In this experiment, both the detection model for process resource usage in high-load environments and the task classification model were deployed on the same server. Detailed information about the experimental environment can be found in Table II.

## B. Experimental design

The experimental design in reference literature[19] involves designing a comprehensive task for integrated tasks, which includes real-time tasks and computation-intensive tasks. The tasks are classified and optimized based on the current CPU load status, and priority scheduling is implemented.

Design a comprehensive task that includes multiple matrix computation task sets, real-time tasks, and periodic tasks. Based on the current CPU load status, the computational tasks are divided into different priority levels. While ensuring the normal operation of real-time tasks and periodic tasks, different computational tasks of varying scales are executed according to the CPU status.

The contrast of resource consumption prediction experiments involves comparing the resource consumption of processes in the future with the results obtained previously using relevant programs. The accuracy of the experiment is determined by comparing the mean squared error and the mean absolute error.

The comparative task classification experiment involves comparing the execution time of a designed task set with normal execution to the execution time of an optimized task set after task classification. By comparing the difference in execution time, conclusions can be drawn.

To ensure the accuracy of the experiment, in the specific design, all CPUs involved in this experiment are configured in single CPU mode. The CPU is subjected to load settings using stress testing tools in Linux, with load levels set at 20%, 50%, and 80%, corresponding to low, medium, and high loads. The overall task execution time is then measured and recorded separately for each of the three load levels.

The experiment is divided into different levels to compare the execution time before and after task classification. This is done to validate the optimization effect of task classification on priority scheduling algorithms.

## C. Model parameters and process resource visualization

In order to ensure the accuracy of the experimental data and obtain more suitable parameters, several comparative experiments were conducted before the formal experimental operation. The parameter chosen for the XGBoost prediction model is "n-estimators." Firstly, the usage of the process resources is displayed, providing the current state of the process. "n-estimators" is a hyperparameter of the XGBoost model, representing the number of weak learners (i.e., trees) to be built. Each weak learner is a decision tree model, and by integrating multiple decision trees, a more powerful model is constructed. Increasing the value of "n-estimators" typically involves using more trees to capture more features and relationships, thereby improving the performance of the

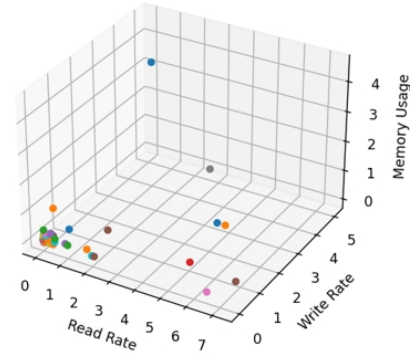


Fig. 3. Process I/O rate and memory usage under high load.

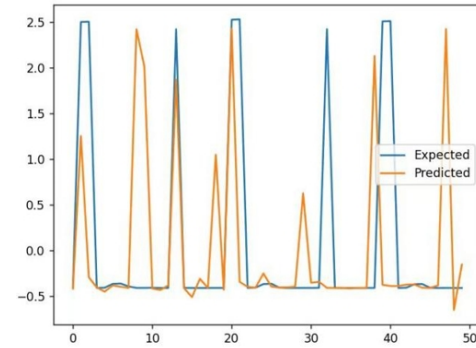


Fig. 4. Load prediction data using traditional methods

model.

In high-load environments, the operating environment of the process is shown in Fig 3., which displays the basic running information of the process. By using this information to collect data, all the collected data has been collected into datasets for further experimental operations. The X-axis represents the process read rate, the Y-axis represents the process write rate, and the Z-axis represents the process memory usage.

## D. Comparison Experiment

**Comparative Analysis of Load Prediction.** The prediction method in the reference literature[20], [21] on experimental methods is simplified in this experiment to make future process resource consumption predictions based only on the current CPU state and the resources consumed by already executed processes. The eBPF program and traditional methods are used to collect process data under the current state, and the mean squared error (MSE) and mean absolute error (MAE) are calculated for both methods. The accuracy can be determined by comparing the sizes of MSE and MAE.

Both Fig 4 and Fig 5 represent the experimental results of load prediction under the same parameter. Fig 4 represents the load prediction data using the traditional method, while Fig 5 represents the load prediction data using the eBPF program.

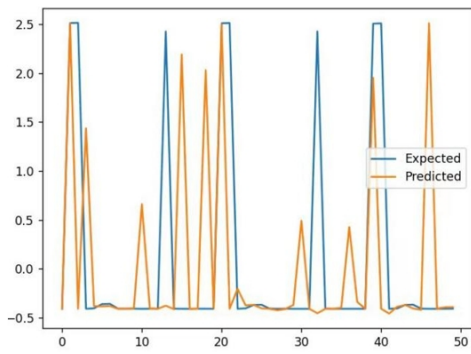


Fig. 5. Load prediction data for eBPF programs

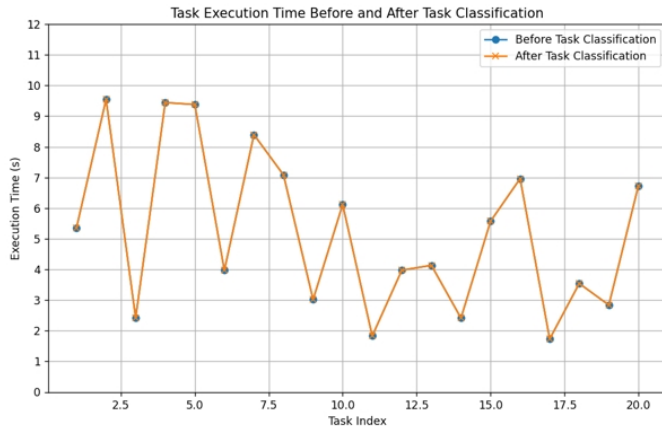


Fig. 6. Preliminary experimental comparison

**Comparison of process execution efficiency after task classification.** Specific experimental references for comparison with system time[22]. To avoid real-time tasks from occupying the CPU for an extended period in standard Linux, a set of periodic tasks with a run-time between 5ms and 80ms were selected for the experiment. These tasks had a maximum utilization rate of no more than 95%, and taking into account the consumption caused by soft and hard interrupts and context switching, the maximum task utilization rate was controlled to approximately 60%. To facilitate calculations, the real-time tasks were designed according to a standard of 2s, and the periodic tasks were controlled with a cycle of 4s per loop with the utilization rate as mentioned previously.

The computational tasks mainly involve matrix calculations, which are designed with three different sizes:  $10 \times 10$ ,  $100 \times 100$ , and  $1000 \times 1000$ . The experimental data is shown in Fig 6.

As the difference between the two is around 109s and cannot be observed in seconds, time units are converted to millisecond level for comparison. In Figure 10, the blue line A represents the normal execution time of the system, and the red line B represents the execution time of the optimized scheduling algorithm after task classification. The horizontal axis compares the time statistics of 10 tasks. According to

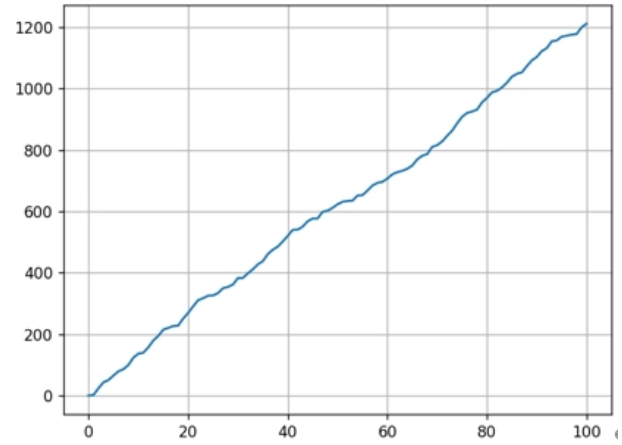


Fig. 7. Millisecond-level comparison

the image, the difference is calculated as 1211.1ms, and the performance is improved by about 1.03%, as shown in Fig 7.

## VI. CONCLUSION

This paper proposes a process prediction and task classification model for high-load environments, which combines data collection based on eBPF, XGBoost predictive model, and classification model. The model utilizes eBPF programs to extract process and CPU-related data from the kernel, leveraging the advantages of fine-grained and real-time analysis compared to traditional tools. The collected data is initially processed and cleaned before being fed into the XGBoost predictive model, which employs an optimized gradient boosting algorithm by adjusting the value of n-estimators to improve experimental results. Following the initial experiments, further data cleansing is performed, retaining only normal processes, and the dataset is inputted into the XGBoost classification model for task classification, utilizing a binary logistic loss function to optimize the model. Experimental results demonstrate that the two models based on eBPF and XGBoost can accurately extract data, predict workload, and classify tasks. Therefore, this model provides an effective and reliable capability for prediction and task classification.

## ACKNOWLEDGMENT

This research is supported by the Key Research and Development Project of Shaanxi Province (No.2024GX-YBXM-110)

This work was supported by ZTE Industry-University-Institute Cooperation Funds under Grant No. IA20221221020.

## REFERENCES

- [1] M. Sun, f. Hang, X. Yin, and j. Gong, "A review of anomaly detection in process-based behavior," *Computer Applications Research*, vol. 25, no. 10, p. 4, 2008.

- [2] C. Y. YU Junpeng, "A practical reinforcement learning framework for automatic radar detection," *ZTE Communications*, vol. 21, no. 3, pp. 22–28, 2023.
- [3] Y. U. Guang-Liang and Y. Meng-Fei, "Response time analysis of embedded systems by taking interrupt and context switch overheads into account," *Journal of Software*, vol. 29, no. 6, p. 18, 2017.
- [4] F. M. David, J. C. Carlyle, and R. H. Campbell, "Context switch overheads for linux on arm platforms," in *Proceedings of the 2007 workshop on Experimental computer science*, 2007, pp. 3–es.
- [5] Y. B. W. D. ZHAO Zipiao, ZHAO Yongli, "Auxiliary fault location on commercial equipment based on supervised machine learning," *ZTE Communications*, vol. 20, no. S1, pp. 7–15, 2022.
- [6] K. Suo, Y. Shi, C.-C. Hung, and P. Bobbie, "Quantifying context switch overhead of artificial intelligence workloads on the cloud and edges," in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, 2021, pp. 1182–1189.
- [7] S. Yin, H. Yang, K. Xu, C. Zhu, S. Zhang, and G. Liu, "Dynamic real-time abnormal energy consumption detection and energy efficiency optimization analysis considering uncertainty," *Applied Energy*, vol. 307, p. 118314, 2022.
- [8] M. Nacchia, F. Fruggiero, A. Lambiase, and K. Bruton, "A systematic mapping of the advancing use of machine learning techniques for predictive maintenance in the manufacturing sector," *Applied Sciences*, vol. 11, no. 6, p. 2546, 2021.
- [9] S. Sharma and C. Azad, "A hybrid approach for feature selection based on global and local optimization for email spam detection," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, 2021, pp. 1–6.
- [10] R. A. A. Habeeb, F. Nasaruddin, A. Gani, I. A. T. Hashem, E. Ahmed, and M. Imran, "Real-time big data processing for anomaly detection: A survey," *International Journal of Information Management*, vol. 45, pp. 289–307, 2019.
- [11] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [12] M. A. Vieira, M. S. Castanho, R. D. Pacifico, E. R. Santos, E. P. C. Júnior, and L. F. Vieira, "Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications," *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–36, 2020.
- [13] D. Scholz, D. Raumer, P. Emmerich, A. Kurtz, K. Lesiak, and G. Carle, "Performance implications of packet filtering with linux ebpf," in *2018 30th International Teletraffic Congress (ITC 30)*, vol. 1. IEEE, 2018, pp. 209–217.
- [14] W. Findlay, A. Somayaji, and D. Barrera, "Bpfbbox: Simple precise process confinement with ebpf," in *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop*, 2020, pp. 91–103.
- [15] B. Brimhall, J. Garrard, C. De La Garza, and J. Coffman, "A comparative analysis of linux mandatory access control policy enforcement mechanisms," in *Proceedings of the 16th European Workshop on System Security*, 2023, pp. 1–7.
- [16] R.-C. Chen, R. E. Caraka, N. E. G. Arnita, S. Pomalingo, A. Rachman, T. Toharudin, S.-K. Tai, and B. Pardamean, "An end to end of scalable tree boosting system," vol. 165, no. 1, 2020, pp. 1–11.
- [17] A. Law, C. Leung, R. Poddar, R. A. Popa, C. Shi, O. Sima, C. Yu, X. Zhang, and W. Zheng, "Secure collaborative training and inference for xgboost," in *Proceedings of the 2020 workshop on privacy-preserving machine learning in practice*, 2020, pp. 21–26.
- [18] S. Ramraj, N. Uzir, R. Sunil, and S. Banerjee, "Experimenting xgboost algorithm for prediction and classification of different datasets," *International Journal of Control Theory and Applications*, vol. 9, no. 40, pp. 651–662, 2016.
- [19] G. N. Dimitrakopoulos, A. G. Vrahatis, V. Plagianakos, and K. Sgarbas, "Pathway analysis using xgboost classification in biomedical data," in *Proceedings of the 10th Hellenic conference on artificial intelligence*, 2018, pp. 1–6.
- [20] J. Yo and A. I. Kistijantoro, "Analyzing fair share fairness of tasks in the linux completely fair scheduler using ebpf," in *2023 10th International Conference on Advanced Informatics: Concept, Theory and Application (ICAICTA)*. IEEE, 2023, pp. 1–6.
- [21] T. Bharguram, P. Rajakumar, and A. A. Banu, "Effectiveness of digital forensic investigation through excavation methods of various linux based tools," *International Journal of Enterprise Network Management*, vol. 15, no. 1, pp. 70–92, 2024.
- [22] X. Zhao, L. Li, and X. Liu, "Instruction extension ensuring time constraints in real time processor," *Beijing Da Xue Xue Bao*, vol. 59, no. 2, pp. 179–188, 2023.